

UNIVERSIDADE
Organização e Arquitetura de Computadores

TRABALHO PRÁTICO 1
CONTROLADOR DE CACHE
Implementação em SystemVerilog

Computer Organization and Design: RISC-V Edition
Capítulo 5, Seção 5.12

2025

GitHub: <https://github.com/arthursignorini/AC3>

SUMÁRIO

1 INTRODUÇÃO	3
2 DESCRIÇÃO DO DESENVOLVIMENTO	3
2.1 Arquitetura Geral	3
2.2 Parâmetros e Decomposição do Endereço	4
2.3 Estrutura de Armazenamento	4
2.4 Máquina de Estados Finitos (FSM)	5
2.5 Políticas Adotadas	5
2.6 Fluxo de Write-Back e Write-Allocate	6
2.7 Dificuldades Encontradas	7
3 RESULTADOS OBTIDOS	7
3.1 Resultados da Simulação	7
3.2 Log Completo da Simulação	8
3.3 Análise de Waveforms	9
3.4 Análise de Desempenho	9
4 CONCLUSÃO	10
5 USO DE INTELIGÊNCIA ARTIFICIAL	10
5.1 Prompts Utilizados	10
5.2 Como a IA foi Utilizada	11
5.3 Responsabilidade Técnica	11

1 INTRODUÇÃO

A hierarquia de memória é um dos pilares fundamentais da arquitetura de computadores modernos. Caches existem para mitigar a disparidade de velocidade entre processador e memória principal, mantendo localmente cópias dos dados acessados com maior frequência. O controlador de cache é o componente de hardware responsável por toda a lógica de acesso, substituição e consistência entre esses níveis da hierarquia.

Este relatório descreve a implementação de um controlador de cache direct-mapped com política write-back e write-allocate em SystemVerilog, conforme especificado no Capítulo 5, Seção 5.12 do livro *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition* (PATTERSON; HENNESSY).

Os objetivos do trabalho são: consolidar o entendimento sobre funcionamento de caches; implementar em nível de hardware os mecanismos de controle; validar

funcionalmente o projeto por meio de simulação automatizada com 26 testes; e documentar as decisões de projeto adotadas.

2 DESCRIÇÃO DO DESENVOLVIMENTO

2.1 Arquitetura Geral

O sistema é composto por dois módulos SystemVerilog. O `cache_controller` é o dispositivo sob teste (DUT) e implementa toda a lógica de controle da cache. O `main_memory` é um modelo comportamental de memória principal com latência configurável, utilizado exclusivamente nas simulações.

A interface do controlador com a CPU usa sinais de handshake (`cpu_req/cpu_ack/cpu_stall`): a CPU apresenta uma requisição e aguarda, sendo travada (stall) enquanto a cache trata um miss. A interface com a memória usa protocolo burst de 4 palavras para carga e descarga de blocos completos.

Quadro 1 – Módulos do projeto

Módulo	Função	Arquivo
<code>cache_controller</code>	FSM principal: hit/miss, write-back, alocação, handshake	<code>rtl/cache_controller.sv</code>
<code>main_memory</code>	Modelo de memória com latência configurável (2 ciclos)	<code>rtl/main_memory.sv</code>
<code>tb_cache_controller</code>	Testbench automatizado com 26 testes em 6 suites	<code>tb/tb_cache_controller.sv</code>

2.2 Parâmetros e Decomposição do Endereço

A cache é parametrizada e configurada por padrão com 1 KB de capacidade, organizada em 64 blocos de 16 bytes. Com mapeamento direto, o endereço de 32 bits é decomposto em três campos, conforme apresentado no Quadro 2.

Quadro 2 – Decomposição do endereço de 32 bits

Campo	Bits	Largura	Descrição
Tag	[31:10]	22 bits	Identifica qual bloco da memória ocupa a linha
Index	[9:4]	6 bits	Seleciona qual das 64 linhas da cache acessar

Offset	[3:0]	4 bits	Byte dentro do bloco (bits [3:2] = offset de palavra)
--------	-------	--------	---

2.3 Estrutura de Armazenamento

Cada linha da cache armazena quatro campos: valid (1 bit), dirty (1 bit), tag (22 bits) e data ($4 \times 32 \text{ bits} = 128 \text{ bits}$). Os campos foram declarados como arrays independentes — e não como packed struct — para garantir compatibilidade com Icarus Verilog, ModelSim, Questa e Verilator:

```

logic cache_valid [0:63];           // bit de validade
logic cache_dirty [0:63];           // bit dirty (write-back)
logic [21:0] cache_tag [0:63];      // tag de 22 bits
logic [31:0] cache_data [0:63][0:3]; // 4 palavras por bloco

```

2.4 Máquina de Estados Finitos (FSM)

O controlador é implementado como uma FSM síncrona de 4 estados com reset assíncrono ativo em nível baixo. A lógica de próximo estado é combinacional (always_comb) e os registradores são atualizados na borda de subida do clock. Os estados são descritos no Quadro 3.

Quadro 3 – Estados da FSM

Estado	Função
IDLE	Aguarda requisição da CPU. Ao receber cpu_req=1, captura endereço, dado e tipo de operação em registradores, e avança para COMPARE_TAG.
COMPARE_TAG	Verifica hit: valid=1 E tag==saved_tag. Em hit, serve a CPU (1 ciclo) e volta ao IDLE. Em miss, decide entre WRITE_BACK (bloco dirty) ou ALLOCATE (bloco limpo).
WRITE_BACK	Escreve as 4 palavras do bloco dirty na memória via burst. Usa burst_cnt (2 bits) para sequenciar. Ao concluir, transita para ALLOCATE.
ALLOCATE	Carrega as 4 palavras do novo bloco da memória via burst. Ao concluir, atualiza valid, dirty=0 e tag, e retorna ao COMPARE_TAG.

As transições de estado são as seguintes:

IDLE	→ cpu_req=1	→ COMPARE_TAG
COMPARE_TAG	→ hit	→ IDLE
COMPARE_TAG	→ miss, dirty=0	→ ALLOCATE
COMPARE_TAG	→ miss, dirty=1	→ WRITE_BACK
WRITE_BACK	→ mem_ack, cnt=3	→ ALLOCATE
ALLOCATE	→ mem_ack, cnt=3	→ COMPARE_TAG

2.5 Políticas Adotadas

O Quadro 4 apresenta as políticas de projeto adotadas e suas respectivas justificativas.

Quadro 4 – Decisões de projeto

Política	Escolha	Justificativa
Organização	Direct-Mapped	Especificada no livro (Seção 5.12). Hardware mínimo, sem comparadores de via.
Escrita (hit)	Write-Back	Atualiza apenas a cache e marca dirty=1. Reduz tráfego com a memória principal.
Escrita (miss)	Write-Allocate	Aloca o bloco antes de escrever. Favorece localidade espacial.
Substituição	Determinística	Direct-mapped: o índice determina univocamente a linha. Sem política LRU/FIFO.
Burst de memória	4 palavras	Tamanho exato do bloco. burst_cnt de 2 bits sequencia as transferências.
Latência memória	2 ciclos	Configurável via parâmetro LATENCY no modelo main_memory.sv.

2.6 Fluxo de Write-Back e Write-Allocate

Em um miss de leitura com bloco dirty, o controlador reconstrói o endereço de destino usando o tag armazenado na linha (não o tag da nova requisição). O sinal wb_base é declarado como assign combinacional:

```
| assign wb_base = {cache_tag[saved_index], saved_index, 4'b0};
```

O write-back emite 4 requisições de escrita sequenciais à memória (burst). Somente após a conclusão do write-back a alocação do novo bloco é iniciada.

Em uma escrita com hit, apenas a palavra endereçada é atualizada na cache e dirty é setado. A memória não é acessada. Em uma escrita com miss (write-allocate), o bloco é primeiro carregado (ALLOCATE), e após retornar ao COMPARE_TAG o hit é confirmado e a escrita é executada normalmente.

2.7 Dificuldades Encontradas

a) Compatibilidade de simuladores: a versão inicial utilizava packed structs com arrays para representar as linhas de cache, construção não suportada pelo Icarus Verilog 12.0. A solução foi separar os campos em quatro arrays independentes, tornando o código portátil para todos os simuladores principais.

- b) Timing de sinais registrados no testbench:** o sinal `cpu_stall` é registrado (atualizado na borda do clock), então só fica estável dois ciclos após a requisição. O testbench inicial amostrava após um único ciclo, capturando o valor de transição. A solução foi aguardar dois ciclos antes de amostrar o stall.
- c) Endereço de write-back:** o endereço de destino durante o write-back deve usar o tag antigo (salvo na linha de cache), não o tag da requisição corrente. Uma confusão entre esses dois valores resultaria em corrupção de dados na memória. O sinal `wb_base` foi declarado como assign combinacional sobre `cache_tag[saved_index]` para evitar esse erro.

3 RESULTADOS OBTIDOS

3.1 Resultados da Simulação

Todos os 26 testes automatizados passaram com sucesso na simulação com Icarus Verilog 12.0. Os testes cobrem todos os cenários obrigatórios da Seção 7 do enunciado, conforme apresentado no Quadro 5.

Quadro 5 – Resultados da simulação (26/26 PASS)

Suite	Descrição	Testes	Status
1 – Read Path	Hit, miss, carregamento de bloco, valid/tag após alocação	5	PASS
2 – Write Path	Write hit, write miss, dirty bit, política write-back verificada	4	PASS
3 – Substituição	Write-back de bloco dirty, preenchimento de 64 blocos, direct-mapped	6	PASS
4 – Consistência	R → W → R, 10 leituras repetidas, conflito de índice com write-back	3	PASS
5 – Casos Limite	Addr=0x0, addr_max=0x3FFC, reset completo, estado vazio	5	PASS
6 – Handshake	cpu_stall durante miss, cpu_ack na conclusão, liberação de stall	3	PASS
Total		26	26/26

3.2 Log Completo da Simulação

A seguir apresenta-se a saída integral do testbench executado com vvp sim/cache_sim:

```
=====
TESTBENCH – Controlador de Cache
1KB, 64 blocos, direct-mapped, write-back, write-alloc
=====
--- Suite 1: Read Path ---
[PASS] 1.1 Read Miss retorna dado correto da memória
[PASS] 1.2 Read Hit retorna mesmo valor
[PASS] 1.3 Read Hit outra palavra do bloco
[PASS] 1.4 valid=1 após miss+alocação
[PASS] 1.5 Tag correta após alocação
--- Suite 2: Write Path ---
[PASS] 2.1 Write Hit – cache atualizada
[PASS] 2.2 Write Hit – dirty=1
[PASS] 2.3 Write-Back – mem não atualizada imediatamente
[PASS] 2.4 Write Miss (write-allocate) – leitura retorna valor
escrito
--- Suite 3: Substituição e Write-Back ---
[PASS] 3.1 Write-Back ocorre ao substituir bloco dirty
[PASS] 3.2 Bloco substituto valid=1
[PASS] 3.2 Tag do bloco substituto correta
Preenchendo todos os 64 blocos...
[PASS] 3.3 Todos os 64 blocos preenchidos (valid=1)
[PASS] 3.4 Direct-Mapped: acesso conflitante substitui bloco
anterior
[PASS] 3.4 Direct-Mapped: bloco substituído não está mais na cache
--- Suite 4: Consistência ---
[PASS] 4.1 R-W-R: leitura reflete escrita
[PASS] 4.2 Valor estável em 10 leituras consecutivas
[PASS] 4.3 Conflito de índice – write-back preservou dado na mem
--- Suite 5: Casos Limite ---
[PASS] 5.1 Acesso ao endereço 0x0
[PASS] 5.2 Acesso ao endereço extremo alto (0x3FFC)
[PASS] 5.3 Reset – todos valid=0
[PASS] 5.4 Reset – todos dirty=0
[PASS] 5.5 Primeiro acesso após reset carrega dado correto
--- Suite 6: Handshake CPU ---
[PASS] 6.1 cpu_stall=1 durante miss
[PASS] 6.2 cpu_ack=1 ao concluir transação
[PASS] 6.3 cpu_stall=0 após conclusão
=====
RESULTADO FINAL
Total : 26 | Passou : 26 | Falhou : 0
*** TODOS OS TESTES PASSARAM ***
=====
```

3.3 Análise de Waveforms

O testbench gera automaticamente o arquivo sim/cache_sim.vcd, que pode ser inspecionado no GTKWave com o comando gtkwave sim/cache_sim.vcd. O Quadro 6 descreve o comportamento esperado nos principais sinais para cada tipo de transação.

Quadro 6 – Comportamento de sinais por tipo de transação (latência mem = 2 ciclos)

Transação	cpu_req	cpu_stall	cpu_ack	mem_req	Ciclos
Read Hit	1 → 0	1 → 0	pulso 1	0	~2
Read Miss (limpo)	1 → 0	1 → 0	pulso 1	burst 4	~11
Read Miss (dirty)	1 → 0	1 → 0	pulso 1	burst 8	~19
Write Hit	1 → 0	1 → 0	pulso 1	0	~2
Write Miss	1 → 0	1 → 0	pulso 1	burst 4	~11

3.4 Análise de Desempenho

Com latência de memória de 2 ciclos e burst de 4 palavras, o custo de cada operação é apresentado no Quadro 7.

Quadro 7 – Latência por operação

Operação	Condição	Ciclos totais
Leitura	Hit	2 (IDLE + COMPARE_TAG)
Leitura	Miss — bloco limpo	11 (2 + 4×2 alocação + 1 serve)
Leitura	Miss — bloco dirty	19 (2 + 4×2 WB + 4×2 aloc + 1)
Escrita	Hit	2
Escrita	Miss — write-allocate, limpo	11
Escrita	Miss — write-allocate, dirty	19

4 CONCLUSÃO

O projeto atingiu integralmente os objetivos propostos. O controlador de cache foi implementado em SystemVerilog com arquitetura direct-mapped, política write-back e write-allocate, interface de handshake com CPU e protocolo burst com a memória, seguindo fielmente a especificação do livro.

A FSM de 4 estados demonstrou-se uma solução clara, modular e robusta. Os 26 testes automatizados — organizados em 6 suites que cobrem exatamente os cenários obrigatórios da Seção 7 do enunciado — passaram integralmente na simulação com Icarus Verilog 12.0, e o waveform VCD gerado permite inspeção visual de qualquer transação.

As dificuldades encontradas foram de natureza prática (compatibilidade de simuladores e timing de sinais registrados) e foram resolvidas e documentadas de forma sistemática, reforçando a compreensão dos mecanismos envolvidos.

Como extensão natural, seria interessante implementar uma cache set-associative de 2 ou 4 vias com política LRU, permitindo comparação quantitativa de taxa de miss entre as organizações para diferentes padrões de acesso.

5 USO DE INTELIGÊNCIA ARTIFICIAL

O desenvolvimento deste trabalho utilizou o modelo Claude Sonnet 4.6 (ANTHROPIC, 2025, acessado via claude.ai) como assistente de desenvolvimento, seguindo o modelo Spec-Driven Development (SDD) exigido no enunciado.

5.1 Prompts Utilizados

Foram utilizados seis prompts pontuais ao longo do desenvolvimento, com o objetivo de obter auxílio em tópicos específicos, como revisão ortográfica, verificação de clareza textual e conferência do código final implementado.

Prompt 1 — Revisão ortográfica e gramatical do relatório.

Pode revisar o texto abaixo e corrigir eventuais erros de ortografia, concordância e pontuação? Não altere o conteúdo técnico, apenas a escrita. [trecho do relatório colado]

A IA revisou o texto fornecido, sinalizando erros de digitação, problemas de concordância verbal e ajustes de pontuação. Nenhuma informação técnica foi alterada; as correções foram limitadas à norma culta da língua portuguesa.

Prompt 2 — Verificação de clareza na descrição da FSM.

Este parágrafo está claro e compreensível para alguém que não conhece o projeto? Se não, sugira como melhorar a redação sem mudar o significado técnico. [parágrafo sobre transições de estado colado]

A IA sugeriu reformulações que tornaram a descrição das transições de estado mais direta, substituindo construções passivas por ativas e eliminando redundâncias. O

grupo avaliou cada sugestão e acatou apenas as que não comprometiam a precisão técnica.

Prompt 3 — Dúvida pontual sobre formatação ABNT.

Na ABNT, a legenda de uma tabela vai acima ou abaixo? E a fonte, como deve ser indicada quando os dados são próprios?

A IA esclareceu que, conforme a NBR 14724, a legenda deve aparecer acima do quadro ou tabela e a fonte abaixo. Com base nessa orientação, o grupo ajustou a formatação dos quadros do relatório.

Prompt 4 — Revisão final de coesão textual.

Leia a seção de conclusão e verifique se as ideias estão bem conectadas e se o texto fecha bem o trabalho. Aponte apenas problemas de coesão ou fluidez.

A IA apontou duas transições abruptas entre parágrafos e sugeriu conectivos mais adequados. O grupo incorporou as sugestões e revisou o parágrafo final para garantir que a conclusão retomasse os objetivos enunciados na introdução.

Prompt 5 – Verificação da lógica do código SystemVerilog.

Pode verificar se esse trecho de código em SystemVerilog faz sentido para a lógica de write-back e write-allocate do controlador de cache? Preciso garantir que a máquina de estados da FSM atenda corretamente aos requisitos de transição do projeto solicitados.

A IA analisou o bloco de código fornecido e confirmou que a lógica condicional e as transições de estado (como a ida de COMPARE_TAG para WRITE_BACK) estavam consistentes. A ferramenta sugeriu apenas pequenas melhorias na indentação e na padronização dos comentários para facilitar a leitura. Nenhuma alteração funcional foi gerada pela IA; a estrutura lógica original do grupo foi integralmente mantida.

Prompt 6 – Sugestão de ajustes e boas práticas no código final.

Aqui está o código final completo do nosso cache_controller.sv e do testbench. Revise o código e sugira apenas ajustes de boas práticas de SystemVerilog, legibilidade e organização (como espaçamento e nomenclatura). Não altere a lógica da máquina de estados (FSM), o mapeamento direto ou as políticas adotadas.

A IA realizou uma análise estática do código final e sugeriu padronizações na nomenclatura de alguns sinais internos, além de recomendar uma separação mais clara entre os blocos sequenciais e combinacionais por meio de comentários. O grupo avaliou as sugestões e aplicou apenas os ajustes cosméticos e de legibilidade, garantindo que o comportamento do hardware e as transições de estados permanecessem estritamente como projetados originalmente.

5.2 Como a IA foi Utilizada

O Quadro 8 detalha o uso da IA por etapa do desenvolvimento.

Quadro 8 – Detalhamento do uso de IA por etapa

Etapas	Uso da IA	Responsabilidade do grupo
Revisão ortográfica	Correção de erros de ortografia, concordância e pontuação	Avaliação e aceitação das correções sugeridas
Clareza textual	Sugestão de reformulações para trechos confusos	Julgamento sobre quais sugestões preservavam o sentido técnico
Formatação ABNT	Esclarecimento de dúvidas pontuais sobre normas	Aplicação das orientações no documento
Coesão textual	Identificação de transições abruptas entre parágrafos	Revisão e reescrita dos trechos indicados
Verificação de lógica	Confirmação da consistência das transições da FSM	Avaliação das observações; manutenção da lógica original
Boas práticas	Sugestão de ajustes de nomenclatura, indentação e comentários	Aplicação seletiva dos ajustes cosméticos; nenhuma alteração funcional

5.3 Responsabilidade Técnica

O grupo desenvolveu e revisou integralmente todo o código e o conteúdo técnico entregue. A IA foi utilizada exclusivamente como ferramenta de apoio à escrita — revisão ortográfica, clareza textual e formatação —, nunca como fonte de conteúdo técnico ou substituta do entendimento do grupo. Todas as decisões de projeto, implementações em SystemVerilog e análises de resultados foram realizadas pelos integrantes, que são capazes de explicar e defender cada escolha implementada.

REFERÊNCIAS

PATTERSON, David A.; HENNESSY, John L. Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition. 2. ed. Waltham: Morgan Kaufmann, 2020.

ANTHROPIC. Claude Sonnet 4.6. Disponível em: <https://www.claude.ai>. Acesso em: 2025.